Week5 미니 프로젝트 보고서

프로젝트명: 미니게임 배틀　학번: 2025311004　이름: 윤정민

A. 기획대로 작동하는가

1. 체크리스트 검증 결과

plan.md 의「이러면 다 된 것」에 작성한 항목을 기준으로 프로그램을 직접 실행하여 기능이 기획대로 작동하는지 확인하였다.

확인 항목	내가 한 조작/입력	기대한 결과	실제 결과	통과?
같은 플레이어 아이디를 받을 수 없다.	P1 과 P2 에 모두 같은이름을 입력한 후 [게임 시작] 클릭	게임 화면으로 넘어가지 않고 경고 메시지 출력	게임이 시작되지 않고 하단에 '같은 플레이어 아이디를 받을 수 없습니다.' 출력	O
P1 은 숫자열 키, P2 는 숫자패드로 입력한다.	① P1 이 상단 숫자열 1 입력 ② P2 가 우측 숫자패드 2 입력	먼저 입력한 플레이어만 답변 가능하고 상대 키는 무시됨	상단 숫자열은 P1, 숫자패드는 P2 가 답변 권한을 얻고 상대 입력은 차단됨	O
오답 또는 시간 초과 시 10 초 동안 다시 입력할 수 없다.	P1 이 오답을 Enter 로 제출한 직후 숫자키를 다시 입력	10 초간 P1 입력 차단 및 남은 시간 표시	'오답! user1 10 초간 입력 불가'와 'P1 패널티: 9.8 초' 등이 표시되고 입력 차단	O
3 판 중 2 판을 이기면 남은 게임 없이 바로 최종 승리가 뜬다.	P1 이 암산과 암기 게임을 연속 승리하여 전체 스코어 2:0 달성	3 게임을 생략하고 최종 승리 화면과 폭죽 출력	5 초간 스코어 표시 후 3 게임이 생략되고 '최종 승리자: user1' 및 폭죽 출력	O

2. 테스트 과정에서 발견한 문제와 조치

테스트 과정에서 우측 숫자패드 입력이 정상적으로 구분되지 않는 문제가 있었다. 초기 구현에서는 Windows 환경의 Tkinter 가 NumLock 활성화 상태에서 숫자패드 입력을 일반 숫자 입력과 유사하게 전달하여 P2 의 입력을 제대로 구분하지 못하였다. 이를 해결하기 위해 main.py 의 get_digits()(L5~L12)에서 event.keycode 를 확인하고 숫자패드의 가상 키코드 96~105 를 직접 판별하도록 수정하였다. 수정 후 상단 숫자열은 P1, 숫자패드는 P2 의 입력으로 정상적으로 구분되었다.

3. 기획 의도와 실제 결과 비교

이 프로그램의 기획 의도는 나와 친구가 한 자리에서 하나의 컴퓨터로 동시에 세 가지 미니게임으로 대결하고, 먼저 2 승을 달성한 사람의 이름과 함께 폭죽을 보여 주는 프로그램을 만드는 것이었다. 실제 테스트 결과 한 키보드의 상단 숫자열과 우측 숫자패드를 두 플레이어가 나누어 사용하여 경쟁할 수 있었으며, 오답 페널티, 게임별 승패, 2 선승 시 남은 게임 생략, 최종 폭죽 화면까지 계획한 기능이 모두 정상적으로 구현되었다.

B. uv 기반 재현성

프로젝트가 기존 개발 환경이 아닌 다른 환경에서도 동일하게 실행되는지 확인하기 위해 제출용 압축 파일을 만든 뒤 별도의 임시 폴더에 압축을 풀어 실행하였다. 압축을 푼 폴더에는 기존 가상환경인 .venv 가 포함되지 않았으며, 프로젝트 실행에 필요한 소스 및 uv 관련 파일만 존재하는 것을 확인하였다.

```
uv run python main.py
```

실행 과정에서 CPython 3.12.15 가 사용되고 새로운 .venv 가 자동으로 생성되는 것을 확인하였다. 이후 720×540 크기의 '미니게임 배틀' Tkinter GUI 창이 정상적으로 실행되었고 아이디 입력과 게임 진행도 정상 작동하였다. 따라서 기존 .venv 를 압축 파일에 포함하지 않아도 uv 가 필요한 실행 환경을 구성하여 프로그램을 재현할 수 있음을 확인하였다.

C. 코드 이해

1. 시작 흐름

프로그램을 실행하면 GUI 윈도우 생성 → App 객체 초기화 → 아이디 입력 화면 생성 → 키 이벤트·타이머 등록 → mainloop 실행 순서로 준비된다.

```
r = tk.Tk()
app = App(r)
```

```
self.root.bind("<Key>", self.on_key)
self.root.after(100, self.tick)
```

main.py L196~L197 에서 tk.Tk()로 GUI 창을 만들고 App(r)을 호출한다. App.__init__()에서는 창 크기를 720×540 으로 지정하고 self.logic, self.active_player 등의 게임 상태 변수를 초기화한다. 이후 build_id_screen()을 호출하여 두 플레이어의 아이디 입력창 self.e1, self.e2 와 [게임 시작] 버튼을 배치한다. bind()는 키보드 입력이 발생할 때 on_key()가 실행되도록 연결하고, after()는 0.1 초마다 tick()이 실행되도록 등록한다. 마지막으로 r.mainloop()가 실행되면서 프로그램은 사용자의 입력을 기다린다.

2. 상호작용 흐름

게임 시작: 사용자가 두 아이디를 입력하고 [게임 시작] 버튼을 누르면 start_game()(main.py L42~L53)이 실행된다. 두 아이디가 비어 있거나 동일한지 확인한 뒤 문제가 없으면 GameLogic(p1, p2)를 생성하고 build_game_screen()으로 게임 화면을 만든 후 첫 번째 암산 문제를 출제한다.

버저 선점과 답 입력: 게임 중 숫자키를 누르면 on_key()가 실행된다. get_digits()는 event.keycode 를 이용하여 상단 숫자열(48~57)은 P1, 우측 숫자패드(96~105)는 P2 의 입력으로 구분한다. 먼저 입력한 플레이어는 start_answer()를 통해 self.active_player 로 지정되고 self.ans_start = time.time()으로 10 초 제한시간이 시작된다.

정답 판정과 페널티: Enter 를 누르면 submit_answer()에서 self.logic.check_answer(ans)로 정답 여부를 확인한다. 정답이면 1 점이 추가되고 한 게임에서 2 점을 먼저 얻으면 승리한다. 오답이거나 10 초 안에 제출하지 못하면 record_wrong()이 호출되어 해당 플레이어의 입력이 10 초간 차단된다.

최종 승리와 폭죽: tick()은 0.1 초마다 제한시간과 게임 사이 대기시간을 확인한다. 한 플레이어가 전체 경기에서 먼저 2 승을 달성하면 남은 게임을 진행하지 않고 VictoryScreen(self.root, w)을 호출하여 승리자 이름과 폭죽 애니메이션을 출력한다.

3. 패키지의 역할

이 프로젝트에서는 uv add 를 이용하여 추가한 외부 서드파티 패키지가 없다. pyproject.toml 의 dependencies = []에서도 이를 확인할 수 있으며 Python 표준 라이브러리만 사용하였다.

모듈	주요 역할	없을 경우
tkinter	GUI 창, 버튼, 입력창, 키 이벤트 및 폭죽 Canvas 구현	GUI 와 시각적 상호작용을 실행할 수 없음

time	답변 제한시간, 10 초 페널티, 게임 사이 대기시간 계산	시간 제한 및 페널티 처리가 불가능
random	암산·암기·소수/합성수 문제와 폭죽 위치/움직임의 난수 생성	무작위 문제 출제와 다양한 폭죽 효과가 불가능
os	TEST_MODE 환경 변수를 확인하여 테스트 시 대기시간 단축	테스트 모드 환경 변수 확인이 불가능

따라서 이 프로젝트는 별도의 외부 패키지 없이 Python 표준 라이브러리만으로 GUI, 문제 생성, 시간 제어 및 애니메이션을 구현하였다.